

5-6장 토큰화·임베딩

[5장 토큰화]

1 단어 및 글자 토큰화

1. 단어 토큰화
2. 글자 토큰화

2 형태소 토큰화

1. 형태소 어휘 사전
2. **KoNLPy**
3. **NLTK**
4. **spaCy**

3 하위 단어 토큰화

1. **바이트 페어 인코딩** (= 다이그램 코딩)
2. **센텐스피스**
3. **토크 나이저 모델 학습**
4. **워드피스**
5. **토크나이저스**

[6장 임베딩]

1 언어 모델

1. 자기회귀 언어 모델
2. 통계적 언어 모델
3. N-gram
4. TF-IDF
5. 단어 빈도
6. 문서 빈도
7. 역문서 빈도
8. TF-IDF

[5장 토큰화]

- 자연어(Natural Language): 인공적으로 만들어진 프로그래밍 언어와 다르게 사람들이 쓰는 언어 활동을 위해 자연히 만들어진 언어
- 자연어 처리(Natural Language Processing, NLP)는 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술
- 자연어 처리 주요 과제
 - 모호성: 맥락에 따라 달라지는 의미
 - 가변성: 사투리, 신조어 등 다양한 언어 변화

- 구조: 구문·의미 분석
- 말뭉치 → 토큰
 - 말뭉치: 자연어 모델을 훈련하고 평가하는 데 사용되는 대규모의 자연어, 언어모델 구축하고 평가하는 데 자주 사용
 - 토큰: 개별 단어 / 문장 부호 등 텍스트 단어 (< 말뭉치)
 - 토큰화: 텍스트를 토큰 단위로 분할 for 컴퓨터의 자연어 이해
- 토큰나이저: 텍스트 문자열을 토큰으로 나누는 알고리즘 또는 소프트웨어
- 토큰화 방법
 - 공백 분할 : 텍스트를 공백단위로 분리해 개별단어로 토큰화
 - 정규표현식 적용 : 정규표현식으로 특정 패턴을 식별해 텍스트 분할
 - 머신러닝 활용: 데이터 세트 기반으로 토큰화하는 방법을 학습한 머신러닝 적용
 - 어휘사전 적용: 사전에 정의된 단어 집합을 토큰으로 사용
 - 어휘사전: 사전에 정의된 단어를 활용해 토큰나이저를 구축하는방법
 - 직접 어휘사전을 구축하기 때문에 없는 단어나 토큰 존재 가능
 - OOV: 사전에 없는 단어
 - 주의점
 - OOV 문제
 - 차원의 저주 : 모든 토큰이나 단어를 벡터화하면 어휘사전에 등장하는 토큰 개수만큼의 차원이 필요, 벡터값이 거의 모두 0의 값을 가지는 희소 (sparse)데이터로 표현 → 문장에서 발생한 토큰들의 순서관계를 잘표현 하지 못함

1 단어 및 글자 토큰화

1. 단어 토큰화

: 텍스트 데이터를 의미있는 단위인 단어로 분리하는 작업

- 띄어쓰기, 문장부호, 대소문자등의 특정구분자를 활용해 토큰화 수행
- 품사 태깅, 개체명 인식, 기계 번역 등의 작업에서 널리 사용되며 가장 일반적인 토큰화 방법

5.1 단어 토큰화

- `split` : 주어진 구분자를 통해 문자열을 리스트 데이터로 나눔. 구분자를 입력하지 않으면 공백기준으로 나눔
- 단어 토큰화는 한국어 접사, 문장 부호, 오타 혹은 띄어쓰기 오류 등에 취약

2. 글자 토큰화

: 띄어쓰기 뿐만 아니라 글자 단위로 문장을 나누는 방식, 비교적 작은 단어 사전 구축 가능

- 언어 모델링과 같은 시퀀스 예측 작업에서 활용

5.2 글자 토큰화

- 리스트 형태로 변환하면 쉽게 수행
- 공백도 토큰으로 나뉨 ($\leftrightarrow$ 단어 토큰화)
- 자소 단위 토큰화 - 자모 라이브러리
 - `jamo.h2j()` : 자모 변환 함수
 - `jamo.j2h2j()` : 한글 호환성 자모 변환 함수

5.3 자소 단위 토큰화

- 단어 단위로 토큰화하는 것에 비해 비교적 적은 크기의 단어 사전 구축 가능 + 접사·문장부호 의미 학습 가능 + OOV 획기적 감소
- but, 개별 토큰 의미 X $\rightarrow$ 토큰 조합 방식으로 문장 생성 / 개체명 인식 등 구현 필요 + 다의어·동음이의어 구별 어려움 + 모델 입력 시퀀스 길이 $\uparrow \Rightarrow$ 연산량 $\uparrow$

2 형태소 토큰화

: 텍스트를 형태소 단위로 나누는 토큰화 방법, 언어의 문법과 구조를 고려해 단어를 분리하고 이를 의미있는 단위로 분류하는 작업

- 형태소: 실제로 의미 가지고 있는 최소의 단위
 - 자립 형태소: 스스로 의미 O, 명사·동사·형용사 등
 - 의존 형태소: 스스로 의미 X, 조사·어미·접사 등

1. 형태소 어휘 사전

: 어휘 사전 중 각 단어의 형태소 정보를 포함하는 사전

- 품사 태깅: 테스트 데이터 형태소 분석 $\rightarrow$ 각 형태소에 해당하는 품사를 캐깅

2. KoNLPy

: 한국어 자연어 처리를 위해 개발된 라이브러리, 명사 추출·형태소 분석·품사 태깅 등의 기능 제공

- 자바 개발 키트(Java Development Kit, JDK) 기반 → 설치 필요
- 지원 형태소 분석기: Okt(OpenKoreanText), 꼬꼬마(Kkma), 코모란(Komoran), 한나눔(Hannanum), 메캅(Mecab)

5.4 Okt 토큰화

- `okt.nouns` : 명사 추출

`okt.phrases` : 구문 추출

`okt.morphs` : 형태소 추출

→ 명사, 어절 구 단위, 형태소 만 추출해 리스트 반환

- `okt.pos` : 품사 태깅

→ 각 단어의 품사 정보 추출 → (형태소, 품사) 형태의 튜플로 구성

- Okt 품사태그

태그	품사	태그	품사
Noun	명사	Punctuation	구두점
Verb	동사	Foreign	외국어
Adjective	형용사	Alpha	알파벳
Determiner	관형사	Number	숫자
Adverb	부사	KoreanParticle	자음/모음
Conjunction	접속사	URL	웹 주소
Exclamation	감탄사	Hashtag	해시태그(#)
Josa	조사	Email	이메일
PreEomi	선어말어미	ScreenName	아이디 표기(@)
Eomi	어미	Unknown	미등록
Suffix	접미사		

5.5 꼬꼬마 토큰화

- `kkma.sentence` : 명사 추출

→ 형태소 분석기에 따라 다른 결과 ⇒ 목적·환경에 맞는 형태소 분석기 사용

- 꼬꼬마 품사 태그

태그	품사	태그	품사
NNG	보통명사	EPH	존칭 선어말 어미
NNP	고유명사	EPT	시제 선어말 어미
NNB	일반 의존 명사	EPP	공손 선어말 어미
NNM	단위 의존 명사	EFN	평서형 종결 어미
NR	수사	EFQ	의문형 종결 어미
NP	대명사	EFO	명령형 종결 어미
VV	동사	EFA	청유형 종결 어미
VA	형용사	EFI	감탄형 종결 어미
VXV	보조 동사	EFR	존칭형 종결 어미
VXA	보조 형용사	ECE	대등 연결 어미
VCP	긍정 지정사	ECS	보조적 연결 어미
VCN	부정 지정사	ECD	의존적 연결 어미
MDN	수 관형사	ETN	명사형 전성 어미
MDT	일반 관형사	ETD	관형형 전성 어미

MAG	일반 부사	XPN	체언 접두사
MAC	접속 부사	XPV	용언 접두사
JKS	주격 조사	XSN	명사파생 접미사
JKC	보격 조사	XSV	동사 파생 접미사
JKG	관형격 조사	XSA	형용사 파생 접미사
JKO	목적격 조사	XR	어근
JKM	부사격 조사	SF	마침표, 물음표, 느낌표
JKI	호격 조사	SE	출입표
JKQ	인용격 조사	SS	따옴표, 괄호표, 줄표
JC	접속 조사	SP	쉼표, 가운데점, 콜론, 빗금
JX	보조사	SO	붙임표(물결, 숨김, 빠짐)
OH	한자	SW	기타기호(논리수학기호, 화폐기호)
OL	외국어	IC	감탄사
ON	숫자	UN	명사추정범주

3. NLTK

: 토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감성 분석 등과 같은 기능 제공

5.6 패키지 및 모델 다운로드

5.7 영문 토큰화

- **punkt** 모델 기반으로 단어·문장 토큰화
 - 단어 토큰나이저: 문장을 입력받아 공백을 기준으로 단어 분리, 구두점 등을 처리해 각각의 단어를 추출해 리스트로 반환
 - 문장 토큰나이저: 문장을 입력받아 마침표 (.), 느낌표 (!), 물음표 (?) 등의 구두점을 기준으로 문장을 분리해 리스트로 반환

5.8 영문 품사 태깅

- NLTK 라이브러리의 품사 태깅 메서드는 토큰화된 문장에서 품사 태깅 수행, 토큰화된 단어 필요

- Averaged Perceptron Tagger 모델: 총 35 개의 품사 태깅 가능
 - Averaged Perceptron Tagger 품사태그

태그	품사	태그	품사
CC	접속사	PDT	관사
CD	기수(Cardinal Number)	POS	소유격 조사
DT	관형사	PRP	인칭 대명사
EX	there의 대명사형태	PRP\$	소유격 대명사
FW	외래어	RB	부사
JN	전치사	RBR	비교급 부사
JJ	형용사	RBS	최상급 부사
JJR	비교급 형용사	RP	전치사 또는 조동사
JJS	최상급 형용사	VB	기본형 동사
LS	목록 표시	VBD	과거형 동사
MD	조동사	VBG	현재 부사형 동사
NN	단수형 명사	VBN	과거 분사형 동사
NNS	복수형 명사	VBP	1인칭 단수 현재형 동사
NNP	단수형 고유 명사와 서수(Ordinal Number)	VBZ	3인칭 단수 현재형 동사
NNPS	복수형 고유 명사	WDT	관계사
SYM	기호	WP	주격 관계대명사
TO	To 부정사	WP\$	소유격 관계대명사
UH	감탄사	WRB	관계부사

4. spaCy

: NLTK와 마찬가지로 자연어 처리 기능 제공

- 빠른 속도와 높은 정확도를 목표로 하는 머신러닝 기반의 자연어 처리 라이브러리 (↔ NLTK)
 - NLTK: 학습 목적으로 자연어 처리에 대한 다양한 알고리즘, 예제 제공
 - spaCy: 효율적인 처리 속도와 높은 정확도 제공이 목표

5.9 spaCy 품사 태깅

- 사전 학습된 모델 기반으로 처리 → `load` 통해 모델 설정 가능
- 객체 지향적으로 구현 → 처리한 결과를 `doc` 객체에 저장 → `doc` 객체는 여러 `token` 객체로 구성, 이 `token` 객체에 대한 정보 기반으로 다양한 자연어 처리 수행
 - `nlp` 인스턴스에 문장 입력 → `doc` 객체 반환 → `token` 객체의 `tag_` / `pos_` 등의 속성에 접근해 값 확인 가능
- `token` 객체: 기본 품사 속성 `pos_`, 세분화 품사 속성 `tag_`, 원본 텍스트 데이터 `text`, 토큰 사이의 공백을 포함하는 텍스트 데이터 `text_with_ws`, 벡터 `vector`, 벡터 노름 `vector_norm` 등의 속성이 포함
- spaCy 품사 태그

pos_ 속성	tag_ 속성	품사	pos_ 속성	tag_ 속성	품사
ADJ	JJ	형용사	NUM	CD	기수(Cardinal Number)
ADJ	JJR	비교급 형용사	PROPN	NNP	고유 명사
ADJ	JJS	최상급 형용사	PROPN	NNPS	복수형 고유 명사
ADP	IN	전치사	PUNCT	-LRB-	왼쪽 괄호
ADV	RB	부사	PUNCT	-RRB-	오른쪽 괄호
ADV	RBR	비교급 부사	PUNCT	,	쉼표
ADV	RBS	최상급 부사	PUNCT	:	콜론
ADV	WRB	관계 부사	PUNCT	.	마침표, 물음표, 느낌표
AUX	MD	조동사	PUNCT	...	생략 부호
CCONJ	CC	접속사	PUNCT	"	여는 따옴표
DET	DT	관사, 한정사	PUNCT	"	닫는 따옴표
DET	PDT	전치사 한정사	PUNCT	HYPH	하이픈
DET	PRP\$	소유격 대명사	PUNCT	NFP	불필요한 문장 부호
DET	WDT	관계 대명사	SYM	\$	통화
DET	WP\$	소유격 관계 대명사	VERB	VB	동사 기본형
INTJ	UH	감탄사	VERB	VBD	과거형 동사
NOUN	NN	명사	VERB	VBG	동명사, 현재분사
NOUN	NNS	복수형 명사	VERB	VBN	과거분사
PART	POS	소유격 조사	VERB	VBP	동사 현재형
PART	RP	부사적 불변화사	VERB	VBZ	3인칭 단수 동사 현재형
PART	TO	To 부정사	X	ADD	이메일
PRON	EX	존재문(there)	X	FW	외래어
PRON	PRP	인칭 대명사	X	LS	목록
PRON	WP	관계 대명사	X	SYM	기호

- 형태소 분석·품사 태깅은 주요 키워드 추출 or 문장의 긍정 / 부정 여부를 판단하는 언어 모델의 성능을 높이는 데도 중요한 역할 수행

3 하위 단어 토큰화

- 언어의 변동성, 맞춤법이나 띄어쓰기가 엄격하게 지켜지지 않은 경우, 신조어나 고유어 오타자 등으로 토큰화 성능이 감소할 수 있음
 - Ex. 시보리도 짱짱해고 허리도 어병하지안구 죠하효
 - 결과: [시, 보리, 짱짱해고, 허리, 도, 어, 병하지안구, 죠, 하, 효]
 - '시보리'라는 외래어 인식 X, 어병하다 인식 X
- 전문 용어나 고유어가 많은 데이터를 처리할 시 약점을 보임
 - Ex. 원문 : 진짜 멋있네요 ! 돈썰날만 하네요.
 - 결과: [' 진짜', '멋있네요 ',' 돈썰날', ' 만' , 하네요 .']
 - → '돈썰내다'라는 신조어를 잘 토큰화하지 못함. 어휘 사전의 크기를 크게 만들

⇒ 즉, 형태소 분석기는 모르는 단어를 적절한 단어로 나누는 것에 취약 → 잠재적으로 어휘 사전의 크기를 크게 만들고 OOV에 대응하기 어렵게 만들 ⇒ 하위 단어 토큰화 필요

- 하위 단어 토큰화: 하나의 단어를 빈번하게 사용되는 하위 단어의 조합으로 나누어 토큰화하는 방법이다.

Ex. 'Reinforcement' → 'Rein' + 'force' + 'ment'

⇒ 단어 길이 ↓ = 처리속도 ↑, OOV 문제, 신조어, 은어, 고유어 등으로 인한 문제 완화

- 바이트 페어 인코딩, 워드 피스, 유니그램 모델 등

1. 바이트 페어 인코딩 (= 다이그램 코딩)

: 하위 단어 토큰화의 한종류, 텍스트 데이터에서 가장 빈번하게 등장하는 글자쌍의 조합을 찾아 부호화하는 압축 알고리즘, 초기에는 데이터 압축을 위해 개발 but, 자연어 처리 분야에 서 하위단어 토큰화 방법으로 사용

- 연속된 글자쌍이 더이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 조합 탐지와 부호화를 반복 → 자주 등장하는 단어는 하나의 토큰으로 토큰화 / 덜 등장하는 단어는 여러 토큰의 조합으로 표현

◦ Ex1. abracadabra

- Step #1 : AracadAra
- Step #2 : ABcadAB
- Step #3 : CcadC

◦ 입력 데이터에서 가장 많이 등장한 글자의 빈도수 측정, 가장 빈도수가 높은 글자쌍 탐색

◦ Step #1 : 원문 abracadabra에서 'ab' 글자쌍이 가장 빈도수 ↑ 입력 데이터에 없는 새로운글자 인 'A'로 치환, 치환되는 새로운 글자는 어떠 한글자를 사용해도 상관 X

◦ Step #2 : 다시 탐색 수행 시 'AracadAra'에서 'ra'글자쌍이 가장 빈도수 ↑ ⇒ 'B'로 치환.

◦ Step #3 : 치환된 데이터는 'ABcadAB' 로 아직 'AB'라는 글자쌍 존재. 치환된 글자 도 글자쌍에 포함 될 수 있으므로 AB를 C로 치환.

⇒ 더 이상 치환할 수 있는 글자쌍 X → 입력데이터 더이상 압축 불가 → ' abracadabra'는CcadC'로 압축

◦ Ex2. 말뭉치에서 바이트 페어 인코딩 적용

- 빈도사전:(1ow,5),(lower,2),(inewest, 6).(widest,3)
어휘사전: ['low', 'lower', 'newest', 'widest']

→ low', lower, newest, widest 각각 5번, 2번, 6번, 3번 등장

- 빈도사전을 바이트 페어인코딩으로 재구성

- 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'i', 'd', 'e', 's', 't', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w']

- 빈도 사전 기준 가장 자주 등장한 글자쌍 탐색

→ 'e', 's' 쌍이 총 9번으로 가장 많이 등장 → 빈도사전에서 'e'와's'를 'es'로 병합하고 어휘사전에 'es' 추가

- 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'es', 't', 6), ('w', 'i', 'd', 'es', 't', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'es']

- 동일한 과정 다시 반복: 빈도수가 가장 높은 'es'와 't' 쌍을 'est' 로 병합 'est'를 어휘사전 추가

- 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'est', 6), ('w', 'i', 'd', 'est', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'es', 'est']

⇒ 총 10번 반복 → 빈도 사전·어휘 사전 생성

- 빈도 사전: ('low', 5), ('low', 'e', 'r', 2), ('n', 'e', 'w', 'est', 6), ('w', 'i', 'd', 'est', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'es', 'est', 'lo', 'low', 'ne', 'new', 'newest', 'wi', 'wid', 'widest']

2. 센텐스피스

: 구글 개발, 오픈소스 하위 단어 토큰라이저 라이브러리

- 바이트페어 인코딩과 유사한 알고리즘 사용 → 입력데이터 토큰화 & 단어사전 생성
- 워드피스, 유니코드 기반의 다양한 알고리즘 지원 → 직접 설정할 수 있는 하이퍼파라미터 제공 → 세밀한 토큰나이징 기능 제공

- 코포라

: 국립국어원, AI Hub 제공, 말뭉치 데이터를 쉽게 사용할 수 있게 제공하는 오픈소스 라이브러리

3. 토큰라이저 모델 학습

5.10 청와대 청원 데이터 다운로드

- `Korpora.load`: 말뭉치 데이터 다운로드 가능
- `corpus`: 총 433,631개의 청원 데이터 저장
- `train`: 학습 데이터 가지고올 수 있음
- `dataset[0]`: 첫번째 청원 데이터
- `petition`: 청원시작일,동의수,제목 등의 정보가 포함

5.11 학습 데이터 세트 생성

- `corpus` - `get_all_texts`: 본문 데이터 세트를 한번에 불러오기 → 하나의 텍스트 파일로 저장 가능

5.12 토크 나이저 모델 학습

- `SentencePieceTrainer` 클래스 - `Train`: 토크 나이저 모델 학습 가능
- 센텐스피스는 문자열 입력 통해 인자들 전달
- 센텐스피스 라이브러리의 주요한 하이퍼파라미터

매개변수	의미
<code>input</code>	말뭉치 텍스트 파일의 경로
<code>model_prefix</code>	모델 파일 이름
<code>vocab_size</code>	어휘 사전 크기
<code>character_coverage</code>	말뭉치 내에 존재하는 글자 중 토크나라이저가 다룰 수 있는 글자의 비율
<code>model_type</code>	토크나라이저 알고리즘 ■ <code>unigram</code> ■ <code>bpe</code> ■ <code>char</code> ■ <code>word</code>
<code>max_sentence_length</code>	최대 문장 길이
<code>unk_id</code>	어휘 사전에 없는 OOV를 의미하는 <code>unk</code> 토큰의 id (기본값: 0)
<code>bos_id</code>	문장이 시작되는 지점을 의미하는 <code>bos</code> 토큰의 id (기본값: 1)
<code>eos_id</code>	문장이 끝나는 지점을 의미하는 <code>eos</code> 토큰의 id (기본값: 2)

- 토크나라이저 모델 학습 완료 → `petition_bpe.model` 파일(학습된 토크나라이저가 저장된 파일) / `petition_bpe.vocab` 파일 생성 (어휘사전이 저장된 파일)
- 센텐스피스 라이브러리는 띄어쓰기나 공백도 특수문자로 취급 → `_` 로 공백 표현
→ Hello world 라는 문장은 `Hello_world` 로 표현되며 토큰화하면 `'_Hello + _Wor + ld'` 로 토큰화 됨

5.13 바이트페어 인코딩 토큰화 학습

- `SentencepieceProcessor` 클래스를 통해 학습된 모델 불러오기 가능

- `tokenizer.load` : 토크나이저 모델 불러오기 메서드
- `encode_as_pieces` : 문장 토큰화
- `encode_as_ids` : 토큰을 정수로 인코딩
→ 정수데이터 = 어휘사전의 토큰에 매핑된 ID값 ⇒ 이 ID값 활용해 자연어 처리 모델 구축
- `decode_ids` / `decode_pieces` : 정수 데이터 → 문자열 데이터 변환

5.14 어휘 사전 불러오기

- `get_piece_size` : 센텐스피스 모델에서 생성된 하위단어의 개수 반환
- `id_to_piece` : 정숫값을 하위 단어로 변환, 하위 단어의 개수 만큼 반복해 하위 단어딕셔너리 구성
- <unk> 토큰 : unknown의 약자로 OOV 발생 시 매핑되는 토큰
- <S>, </s> : 문장의 시작지점과 종료지점을 표시하는 토큰
- 센텐스피스 라이브러리는 설정값에 따라 학습 방법이나 어휘사전 크기가 달라질 수 있음

4. 워드피스

: 바이트페어 인코딩 토크나이저와 유사한 방법으로 학습되지만, 빈도 기반 아닌 확률기반으로 글자쌍을 병합

- 학습 과정에서 확률적인 정보 사용, 모델이 새로운 하위 단어를 생성할 때 이전 하위 단어와 함께 나타날 확률을 계산해 가장 높은 확률을 가진 하위단어 선택
→ 선택된 하위단어는 이후에 더 높은 확률로 선택될 가능성 ↑
⇒ 모델이 좀 더 정확한 하위단어로 분리 가능
- 글자쌍 병합 점수 수식

$$score = \frac{f(x,y)}{f(x)f(y)}$$

- `f` : 빈도(frequency)를 나타내는 함수
- `x,y` : 병합하려는 하위 단어
- `f(x,y)` : x와 y가 조합된 글자쌍의 빈도 = xy글자쌍의 빈도
- `score` : x와 y를 병합하는 것이 적절한지를 판단하기 위한 점수

- 워드피스의 어휘사전 구축 방법

- 빈도사전 내의 모든 단어를 글자 단위로 나눔

- 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'i', 'd', 'e', 's', 't', 3)
 - 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w']

- 가장 빈번하게 등장하는 쌍은 'e'와 's'로 9번 등장. 'e'는 17번, 's'는 9번 등장

→ 점수는 $9/(17*9) = 0.06$

'i'와 'd' 쌍은 3번 밖에 등장하지 않았지만 'i'와 'd'가 각각 3번씩 등장

→ 점수는 $3/(3*3) = 0.33$

∴ 'i'와 'd' 쌍을 병합

- 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'id', 'e', 's', 't', 3)
 - 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'id']

- 동일한 과정을 반복해 각 글자쌍에 대한 점수를 계산

'l'과 'o' 쌍 3번 등장, 'l'과 'o' 각각 7번 등장

→ 점수는 $7/(7*7) = 0.14$

∴ 'l'과 'o' 글자쌍 병합

- 빈도 사전: ('lo', 'w', 5), ('lo', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'id', 'e', 's', 't', 3)
 - 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'id', 'lo']

⇒ 위 과정을 반복해 연속된 글자쌍이 더이상 나타나지 않거나 정해진 어휘사전 크기에 도달할 때까지 학습

5. 토크나이저스

- 정규화와 사전토큰화를 제공

- 정규화

- 일관된 형식으로 텍스트를 표준화, 모호한 경우 방지하기 위해 일부 문자를 대체하거나 제거하는 등의 작업 수행
 - 불필요한 공백제거, 대소문자 변환, 유니코드 정규화, 구두점 처리, 특수문자 처리 등을 제공

- 사전토큰화

- 입력 문장을 토큰화 하기 전에 단어와 같은 작은 단위로 나누는 기능 제공
- 공백 혹은 구두점을 기준으로 입력 문장을 나눠 텍스트 데이터를 효율적으로 처리, 모델 성능 향상

5.15 워드피스 토큰라이저 학습

- 토큰라이저 `Tokenizer` 로 워드피스 `Wordpiece` 모델 불러옴

→ 정규화 방식 / 사전토큰화 방식 설정

- 정규화 방식: `normalizers` 모듈에 포함된 클래스 불러와 시퀀스 형식으로 인스턴스 전달
 - Ex. NFD 유니코드 정규화(NFD), 소문자 변환(Lowercase)
 - 정규화 클래스

이름	설명
<code>NFD()</code>	NFD 유니코드 정규화
<code>NFKD()</code>	NFKD 유니코드 정규화
<code>NFC()</code>	NFC 유니코드 정규화
<code>NFKC()</code>	NFKC 유니코드 정규화
<code>Lowercase()</code>	입력 문장의 모든 대문자를 소문자로 변환
<code>Strip()</code>	입력 문장의 양 끝에 있는 공백 제거
<code>StripAccents()</code>	모든 억센트 기호 제거
<code>Replace()</code>	문자 혹은 정규 표현식을 기반으로 입력 문장 변환
<code>BertNormalizer()</code>	BERT 모델에 사용된 정규화 적용
<code>Sequence()</code>	여러 개의 정규화 모듈을 병합해 순서대로 수행

- 사전토큰화 방식: `pre_tokenizers` 모듈에 포함된 클래스 불러와 적용
 - Ex. 공백과 구두점을 기준으로 분리
 - 사전토큰화 클래스

클래스	설명
Sequence()	여러 개의 사전 토큰화 모듈을 병합하여 순서대로 수행
ByteLevel()	OpenAI에서 GPT-2를 학습할 때 사용한 방법으로 문자열을 그래픽 문자열로 매핑하고 공백을 기준으로 나눈다.
CharDelimiterSplit()	문자 기준으로 문자열을 나눈다.
Digits()	모든 형태의 숫자 표현을 기준으로 문자열을 나눈다.
Punctuation()	구두점을 기준으로 입력 문자열을 나눈다.
Metaspace(replacement="_")	Whitespace 기준으로 사전 토큰화를 진행하고, replacement로 Whitespace를 치환한다. ▪ 기본값: _(U+2581)
Whitespace()	단어 경계(공백과 구두점)를 기준으로 사전 토큰화를 진행한다. ▪ <code>\w+!["^\\w\\s]+</code> 정규 표현식을 적용한다.
WhitespaceSplit()	공백 문자를 기준으로 입력 문자열을 나눈다.
Split(pattern, behavior)	정규표현식(pattern)과 동작(behavior)을 기준으로 문자열을 나눈다. ▪ behavior - removed: 삭제 - isolated: 개별 토큰 처리 - merged_with_previous: 이전 문자열과 결합해 토큰 처리 - merged_with_next: 다음 문자열과 결합해 토큰 처리 - contiguous: 다음 문자열이 토큰화되지 않아도 결합해 토큰 처리

- **train** : 훈련 메서드, 학습에 사용하려는 데이터 세트 경로 전달
- **save** : 저장 메서드, 학습 결과 저장

⇒ 워드피스 모델의 학습 완료 시 **petition_wordpiece.json** 파일 생성됨 → JSON 형태로 저장, 정규화 및 사전토큰화 등의 메타데이터와 함께 어휘사전 저장

5.16 워드피스 토큰화

- 모델결과가 저장된 **petition_wordpiece.json** 파일 불러와 **Tokenizer** 객체 생성
- **encode** : 인코딩 메서드, 문장 토큰화
- **encode_batch** : 인코딩 배치 메서드, 여러문장 한 번에 토큰화
→ 인코딩 메서드 / 인코딩 배치 메서드 모두 워드피스 토큰라이저를 통해 문장을 토큰화 & 각 토큰의 색인번호 반환
- **tokens** : 토큰 속성, 토큰화 된 데이터 값 확인
- **ids** : 토큰 정수 속성, 인코딩된 문장의 ID 값 출력
- **decode** : 디코딩 메서드, 정수 인코딩 된 결과를 다시 문장으로 디코딩해 출력
- 최근 연구 동향 : 더 큰 말뭉치를 사용해 모델을 학습하고 OOV의 위험을 줄이기 위해 하위단어 토큰화 활용
- 바이트페어 인코딩 : 바이트 수준에서 토큰화
- 유니그램: 크기가 큰 어휘사전에서 덜 필요한 토큰을 제거하며 학습

[6장 임베딩]

- 컴퓨터는 텍스트 자체를 이해할 수 없으므로, 텍스트를 숫자로 변환하는 텍스트 벡터화 과정이 필요
- 텍스트 벡터화 : 텍스트를 숫자로 변환하는 과정
 - 원핫인코딩 (One-Hot Encoding)
 - 문서에 등장하는 각 단어를 고유한 색인 값으로 매핑한 후, 해당 색인 위치를 1로 표시하고 나머지 위치는 모두 0으로 표시하는 방식
 - 같은 단어가 여러번 등장하더라도 1과 0으로 표현
 - Ex. 'I like apples' 문장과 'I like bananas' 문장
→ 띄어쓰기 기준으로 토큰화, 단어를 고유한 색인 값으로 매핑

색인	0	1	2	3
토큰	I	like	apples	bananas

▪ I like apples: [1, 1, 1, 0]

▪ I like bananas: [1, 1, 0, 1]

- 빈도 벡터화 (Count Vectorization)

- 문서에서 단어의 빈도수를 세어 해당 단어의 빈도를 벡터로 표현
- apples라는 단어가 총 4번 등장 → 해당 단어의 벡터값은 4

⇒ 단어나 문장을 벡터 형태로 변환하기 쉽고 간단하지만, 벡터의 희소성이 크다는 단점이 있음

- 말뭉치 내에 존재하는 토큰 개수만큼 벡터의 차원을 가져야 하지만, 입력 문장 내에 존재하는 토큰의 수는 그에 비해 현저히 적음 → 컴퓨팅 비용의 증가, 차원의 저주 같은 문제 발생 가능
- 텍스트의 벡터가 입력 텍스트의 의미를 내포하고 있지 않음 → 두 문장이 의미적으로 유사해도 벡터는 유사하게 나타나지 않을 수 있음
- Ex. 'i scream for ice cream'과 'ice cream for i scream'에 원-핫 인코딩 / 빈도 벡터화를 적용하면 둘 다 [1, 1, 1, 1, 1] 벡터 반환

⇒ 문제를 해결하기 위해 워드 투 벡터(Word2Vec)나 패스트 텍스트(fastText) 등과 같이 단어의 의미를 학습해 표현하는 워드 임베딩 기법(Word Embedding) 사용

- 워드 임베딩 기법

: 단어를 고정된 길이의 실수 벡터로 표현하는 방법

- 단어의 의미를 벡터 공간에서 다른 단어와의 상대적 위치로 표현해 단어 간의 관계를 추론
- 고정된 임베딩을 학습하기 → 다의어나 문맥 정보를 다루기 어렵다는 단점
⇒ 인공 신경망을 활용해 동적 임베딩(Dynamic Embedding) 기법 사용

1 언어 모델

: 입력된 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델

- 주어진 문장을 바탕으로 문맥을 이해하고, 문장 구성에 대한 예측을 수행
- 자동 번역, 음성 인식, 텍스트 요약 등 다양한 자연어 처리 분야에서 활용

$P(\text{만나서 반갑습니다.}) = 0.081$		$P(\text{서울특별시}) = 0.59$	
Input : 안녕하세요	...	Input : 대한민국 _ _ _ 강남구	...
$P(\text{아니오, 괜찮습니다.}) = 0.000001$		$P(\text{아이스 아메리카노}) = 0.000001$	

- '안녕하세요'라는 문장 뒤에 어떤 문장이 어떤 확률로 등장할지 계산하는 것은 어려움
→ 이는 비지도 학습 문제로 모델 스스로 문장 등장 확률을 계산 필요
- 또한 '안녕하세요' 다음에 올 수 있는 문장은 사실상 무한에 가깝기 때문에 문장 단위로 확률을 계산하는 것은 불가능
→ 문장 전체를 예측하는 방법 대신, 하나의 토큰 단위로 예측하는 방법인 자기회귀 모델이 고안됨

1. 자기회귀 언어 모델

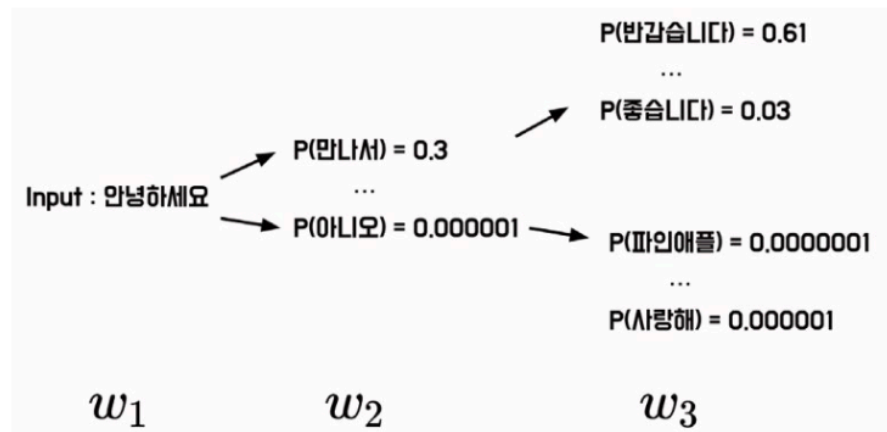
- 입력된 문장들의 조건부 확률을 이용해 다음에 올 단어를 예측
- **조건부 확률**: 이전 단어들의 시퀀스가 주어졌을 때, 다음 단어의 확률을 계산하는 것
 - 이를 위해 이전에 등장한 모든 토큰의 정보를 고려, 문장의 문맥 정보를 파악하여 다음 단어를 생성
 - 다음 단어는 다시 이전 단어를 기반으로 예측이 이루어지며, 이 과정이 반복
 - 언어 모델의 조건부 확률 수식

$$P(w_t | w_1, w_2, \dots, w_{t-1}) = \frac{P(w_1, w_2, \dots, w_t)}{P(w_1, w_2, \dots, w_{t-1})}$$

- **조건부 확률의 연쇄법칙:** 언어 모델에서 조건부 확률을 계산하기 위해 이전에 등장한 단어 시퀀스를 기반으로 다음 단어의 확률을 계산
 - 조건부 확률의 연쇄법칙을 적용한 언어모델의 조건부확률 수식

$$P(w_t | w_1, w_2, \dots, w_{t-1}) = P(w_1)P(w_2 | w_1) \dots P(w_t | w_1, w_2, \dots, w_{t-1})$$

- Ex. [조건부 확률의 연쇄법칙을 통한 자기회귀 언어모델 구조] ‘안녕하세요’ 다음에 등장할 수 있는 문장을 조건부 확률의 연쇄 법칙 방법으로 시각화



- 자기회귀 언어 모델에서는 각 시점에서 다음에 올 토큰을 예측하는 것이 중요
 - 이를 위해 입력 토큰 w_1 을 바탕으로 모델은 다음 토큰 w_2 가 등장할 확률 $P(w_2 | w_1)$ 예측
 - 그 다음 모델의 출력값 w_2 가 다시 입력값이 되어 모델은 확률 $P(w_3 | w_1, w_2)$ 예측
 - 이러한 방식으로 모델의 출력값이 다시 모델의 입력값으로 사용되는 특징 때문에 자기회귀라는 이름이 붙음
- 자기 회귀 언어 모델은 시점 별로 다음에 올 토큰을 예측하는 것이므로, 토큰 분류 문제로 정의할 수 있음
- 자기 회귀 모델은 이전에 등장한 모든 토큰 정보를 활용해 입력된 문장의 문맥 정보를 파악하고 다음 토큰을 예측 → 언어 모델은 문장 전체의 확률을 계산하고, 이를 이용해 다음 단어를 예측

2. 통계적 언어 모델

- 언어의 통계적 구조를 이용해 문장이나 단어의 시퀀스를 생성하거나 분석
- 시퀀스에 대한 확률 분포를 추정해 문장의 문맥을 파악해 다음에 등장할 단어의 확률을 예측
- 일반적으로 통계적 언어 모델은 Markov Chain을 이용해 구현
 - 마르코프 체인 : 빈도 기반의 조건부 확률 모델 중 하나, 이전 상태와 현재 상태 간의 전이 확률을 이용해 다음 상태를 예측
- 빈도 기반의 조건부 확률 모델 : 주어진 데이터에서 각 변수가 발생한 빈도수를 기반으로 확률 계산
 - Ex. if, 말뭉치에 '안녕하세요'라는 문장이 1,000번 등장, 이어서 '안녕하세요 만나서'가 700번, '안녕하세요 반갑습니다'가 100번 등장 할 경우의 빈도의 기반 조건부 확률 수식

$$P(\text{만나서} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 만나서})}{P(\text{안녕하세요})} = \frac{700}{1000}$$

$$P(\text{반갑습니다} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 반갑습니다})}{P(\text{안녕하세요})} = \frac{100}{1000}$$

- 단어의 순서와 빈도에만 기초해 문장의 확률을 예측 → 문맥을 제대로 파악하지 못하면 불완전하거나 부적절한 결과 생성할 가능성 있음
- 데이터의 희소성 문제: 한 빈도 등장한 적 없는 단어나 문장에 대해서는 정확한 확률을 예측하기 어려움
- But 통계적 언어 모델은 기존에 학습한 텍스트 데이터에서 패턴을 찾아 확률 분포를 생성
 - 이를 이용해 새로운 문장을 생성할 수 있으며, 다양한 종류의 텍스트 데이터 학습 가능
 - 대규모 자연어 데이터를 처리하는데 효과적이며, 딥러닝 등의 인공지능 기술이 발전하면서 더욱 강력한 모델을 구현
 - 최근 연구되는 자연어 처리 기법은 언어 모델을 활용해 가중치를 사전학습함
 - Ex. 트랜스포머 모델에 기반한 언어 모델인 GPT, BERT

GPT

- Raw Sentence: GPT는 OpenAI에서 개발한 인과적 언어 모델입니다.
- Input: GPT는 OpenAI에서 개발한
- Prediction-1: 인과적
- Prediction-2: 언어 모델입니다.

BERT

- Raw Sentence: BERT는 Google에서 개발한 마스킹된 언어 모델입니다.
- Input: Bert는 [MASK]에서 개발한 [MASK] 언어 모델입니다.
- Prediction: Google, 마스킹된

3. N-gram

- 가장 기초적인 통계적 언어 모델, 텍스트에서 N개의 연속된 단어 시퀀스를 하나의 단위로 취급하여 특정 단어 시퀀스가 등장할 확률 추정
- 입력 텍스트를 하나의 토큰 단위로 분석하지 않고 N개의 토큰을 묶어서 분석
→ 이때, 연속된 N개의 단어를 하나의 단위로 취급하여 추론하는 모델이며, N이 1일 때는 유니그램(Unigram), N이 2일 때는 바이그램(Bigram), N이 3일 때는 트라이그램(Trigram)으로 부름, $N \geq 4$ 일 때는 N-gram이라고 함

Unigram : <u>안녕하세요</u> 만나서 진심으로 반가워요	안녕하세요. 만나서. 진심으로. 반가워요
Bigram : <u>안녕하세요 만나서</u> 진심으로 반가워요	안녕하세요 만나서. 만나서 진심으로. 진심으로 반가워요
Trigram : <u>안녕하세요 만나서 진심으로</u> 반가워요	안녕하세요 만나서 진심으로. 만나서 진심으로 반가워요

- N-gram 언어 모델은 모든 토큰을 사용하지 않고 N-1개의 토큰만을 고려해 확률 계산.
 - N-gram 모델에서 t 번째 토큰의 조건부 확률을 계산하는 수식

$$P(w_t | w_{t-1}, w_{t-2}, \dots, w_{t-N+1})$$

6.1 N-gram

- 통계적 언어 모델은 상대적으로 구현이 쉬움. 파이썬으로 구현한 N-gram과 NLTK 라이브러리에서 지원하는 N-gram 함수는 동일한 출력값을 반환

- N-gram은 작은 규모의 데이터셋에서 연속된 문자열 패턴을 분석하는데 큰 효과를 보이며, 관용적 표현 분석에서도 활용됨
- N-gram은 시퀀스에서 연속된 n개의 단어를 추출하므로 단어의 순서가 중요한 자연어 처리 작업 및 문자열 패턴 분석에 활용

4. TF-IDF

- 텍스트 문서에서 특정 단어의 중요도를 계산하는 방법으로, 문서 내에서 단어의 중요도를 평가하는데 사용되는 통계적인 가중치 의미.
- TF-IDF는 BoW(Bag-of-Words)에 가중치를 부여하는 방법
- BoW는 문서나 문장을 단어의 집합으로 표현하는 방법으로, 문서나 문장에 등장하는 단어의 중복을 허용해 빈도 기록
- 원-핫 인코딩은 단어의 등장 여부를 판별해 0과 1로 표현하는 방식이지만, BoW는 등장 빈도를 고려해 표현
 - ['That movie is famous movie', 'I like that actor', 'I don't like that actor']
말뭉치를 BoW로 벡터화

	I	like	this	movie	don't	famous	is
This movie is famous movie	0	0	1	2	0	1	1
I like this movie	1	1	1	1	0	0	0
I don't like this movie	1	1	1	1	1	0	0

- BoW를 이용해 벡터화하는 경우 모든 단어는 동일한 가중치를 가짐. BoW 벡터를 활용해 영화 리뷰의 긍/부정 분류 모델을 만든다고 가정한다면 높은 성능 얻기 어려움
- 'I like this movie'와 'I don't like this movie' 문장은 'don't'라는 단어가 문장 분류에 결정적인 역할을 하지만, 'don't'라는 단어가 자주 등장하지 않으므로 토큰화나 분류 모델 진행 시 해당 데이터가 무시될 수 있음

5. 단어 빈도

- 문서 내에서 특정 단어의 빈도수를 나타내는 값
- Ex. 3개의 문서에서 'movie'라는 단어가 4번 등장한다면 해당 단어의 TF 값은 4가 됨

$$TF(t, d) = count(t, d)$$

- 앞선 BoW 벡터 표현 방법 과 같이 문서 내에서 등장한 빈도수를 계산하며, 해당 단어의 상대적 중요도를 측정하는데 사용됨
- TF 값이 높을 수록 해당 단어가 특정 문서에서 중요한 역할을 한다고 생각할 수도 있지만, 단어 자체가 특정 문서 내에서 자주 사용되는 단어이므로 전문 용어나 관용어로 간주할 수 있음
- TF는 단순히 단어의 등장 빈도수를 계산하기 때문에 문서의 길이가 길어질수록 해당 단어의 TF 값도 높아질 수 있음

6. 문서 빈도

- 한 단어가 얼마나 많은 문서에 나타나는지를 의미
- 특정 단어가 많은 문서에 나타나면 문서 집합에서 단어가 나타나는 횟수를 계산
 - Ex. 3개의 문서에서 'movie'라는 단어가 4번 등장한다면 해당 단어의 DF 값은 3가 됨

$$DF(t,D) = count(t \in d: d \in D)$$

- DF는 단어가 몇 개의 문서에 등장하는지 계산하기 때문에 DF 값이 높으면 특정 단어가 많은 문서에서 등장한다고 볼 수 있음. 그 단어는 일반적으로 널리 사용되며, 중요도가 낮을 수 있음
- 반대로 DF 값이 낮다면 특정 단어가 적은 수의 문서에만 등장한다는 뜻임. 그러므로 특정한 문맥에서만 사용되는 단어일 가능성이 있으며, 중요도가 높을 수 있음.

7. 역문서 빈도

- 전체 문서 수를 빈도로 나눈 다음에 로그를 취한 값, 문서 내에서 특정 단어가 얼마나 중요한 지를 나타냄
- 문서 빈도가 높을 수록 해당 단어가 일반적이고 상대적으로 중요하지 않다는 의미 → 문서 빈도의 역수를 취하면 단어의 빈도수가 적을 수록 IDF 값이 커지게 보정하는 역할
- IDF는 분모의 DF 값에 1을 더한 값을 사용 → 특정 단어가 한 번도 등장하지 않는다면 분모가 0이 되는 경우가 발생

$$IDF(t,D) = \log\left(\frac{count(D)}{1 + DF(t,D)}\right)$$

- 추가로 IDF는 로그를 취함. 전체 문서 수를 빈도로 나눈 값을 사용한다면 너무 큰 값이나 올 수 있음. 10, 000개의 문서에서 특정 단어가 1번만 등장한다면 IDF 값은 5,000이 된다. 이런 문제점을 방지하고자 로그를 취해 정교한 가중치를 얻는다.

8. TF-IDF

- 앞선 문서 빈도와 역문서 빈도를 곱한 값으로 사용

$$TF-IDF(t, d, D) = TF(t, d) \times IDF(t, d)$$

- 문서 내에 단어가 자주 등장하지만, 전체 문서 내에 해당 단어가 적게 등장한다면 TFIDF 값은 커짐. 전체 문서에서 자주 등장할 확률이 높은 관사나 관용어 등의 가중치는 낮아짐
- `scikit-learn` 패키지를 사용해 TF-IDF를 계산
- TF-IDF 클래스

```
tfidf_vectorizer = sklearn.feature_extraction.text.TfidfVectorizer(
    input="content",
    encoding="utf-8",
    lowercase=True,
    stop_words=None,
    ngram_range=(1, 1),
    max_df=1.0,
    min_df=1,
    vocabulary=None,
    smooth_idf=True,
)
```

◦ 입력값(input)

- 입력될 데이터의 형태
- 기본값: Content(문자열/바이트 형태)
- 파일 객체를 사용한다면 file로 입력
- 파일 경로를 사용하는 경우 filename으로 입력

◦ 인코딩(encoding)

- 바이트 혹은 파일을 입력값으로 받을 경우 사용할 텍스트 인코딩 값
- 소문자 변환(lowercase)
- 입력받은 데이터를 소문자로 변환할지 여부

- True = 모든 입력 텍스트를 소문자로 변환
- 불용어(stop_words)
 - 분석에 도움이 되지 않는 의미 없는 단어들
 - 입력받은 단어들은 단어 사전에 추가되지 않음
- N-gram *+l (ngram_range)
 - 사용할 N-gram의 범위
 - (최솟값, 최댓값) 형태로 입력
 - (1, 1): 유니그램
 - (1, 2): 유니그램&바이그램
- 최댓값 문서 빈도(max_df)
 - 전체 문서 중 일정 횟수 이상 등장하는 단어는 불용어로 처리
 - 정수 입력 => 해당 등장 횟수를 초과해 등장하는 단어를 불용어 처리
 - 1 이하의 실수 입력 => 해당 비율을 초과해 등장한 단어를 불용어 처리
- 최솟값 문서 빈도(min_df)
 - 전체 문서 중 일정 횟수 미만으로 등장한 단어를 불용어 처리
 - 최댓값 문서 빈도와 동일한 패턴으로 입력
- 단어사전(vocabulary)
 - 미리 구축한 단어사전이 있다면 해당 단어 사전을 사용
 - 입력하지 않는다면 TF-IDF 학습 시 자동으로 구축
- IDF 분모처리(smooth_idf)
 - IDF 계산 시 분모에 1을 더함

6.2 TF- DF 계산

- **fit** 메서드
 - fit 메서드를 통해 학습해야 데이터의 변환/추론이 가능
 - 준비된 말뭉치를 학습
- **transform** 메서드
 - 객체의 학습이 완료되면 transform 메서드를 이용해 데이터 변환을 수행
 - 학습과 변환을 동시에 수행할 수도 있음

- `toarray` 메서드
 - TF-IDF를 넘파이 배열로 변환
 - (문서 수)*(단어 수)의 형태를 가짐
 - 각 행은 하나의 문서에 해당. 각 열은 단어를 의미
- `vocabulary_` 속성
 - TF-IDF에 사용된 단어 사전
 - 딕셔너리의 키: 고유한 단어
 - 딕셔너리의 값: 해당 단어에 대한 색인 값
 - 점수가 가장 높은 값을 세 개만 추려 색인으로 정리
 - 사전으로 매핑
 - 문서마다 중요한 단어만 추출할 수 있으며, 벡터값을 활용해 문서 내 핵심 단어를 추출할 수 있음
- but 빈도 기반 벡터화는 문장의 순서나 문맥을 고려하지 않음 → 문장 생성과 같이 순서가 중요한 작업에는 부적합, 벡터가 해당 문서 내의 중요도를 의미할 뿐, 벡터가 단어의 의미를 담고 있지는 않음